

Reviewer Pack — Minimal Symmetry Groups and Circuit Depth Inequality

Entry 2dafccccf1e2 · INCONCLUSIVE

SEC autonomous research engine — attribution: Ludovico Kubler

2026-06-06 17:26:27 UTC

Minimal Symmetry Groups and Circuit Depth Inequality

Entry ID: 2dafccccf1e2 **Verdict:** INCONCLUSIVE **Recorded:** 2026-06-06 17:26:27 UTC

1. Conjecture

Field A (mathematical branch): Group Theory (Symmetry Groups) **Field B** (complexity object): Boolean Circuit Complexity (Circuit Depth)

Statement:

For every Boolean circuit C with depth d , the order of its symmetry group G_C is upper-bounded by 2^d .

Rationale (proposer's reasoning):

The symmetry group captures the symmetries that preserve the circuit's functionality, and if a circuit can be simplified or transformed into an equivalent circuit with fewer gates without altering its depth, this implies a larger symmetry group. This conjecture would provide a direct link between symmetry invariants and circuit complexity.

Taxonomy category: GROUP_THEORY_x_BOOLEAN_CIRCUIT_COMPLEXITY (status at proposal time:)

2. Pre-registration (Popper-style)

Hash (SHA-256 prefix): 3eb1f4547e5ad61a

This hash commits to the conjecture statement, acceptance criterion, and seed list **before** the empirical test runs. Tampering with the test or its data after this hash was computed would be detectable.

Acceptance criterion:

For a Boolean circuit with depth d , if the order of its symmetry group G_C is less than or equal to 2^d for at least 80% of the seeds and the average order does not exceed $2^{(d+1)}$, then the conjecture is supported.

3. Barrier filter (F1)

Final: PASS

Barrier	Verdict	Confidence	LLM ₁	LLM ₂
RELATIVIZATION	UNCERTAIN	1.00	HITS	SAFE
NATURAL_PROOFS	SAFE	0.50	UNCERTAIN	UNCERTAIN
ALGEBRIZATION	SAFE	0.50	UNCERTAIN	UNCERTAIN

Barrier	Verdict	Confidence	LLM ₁	LLM ₂
KARP_LIPTON	SAFE	0.50	UNCERTAIN	UNCERTAIN

4. Novelty audit

Verdict: NOVEL against 0 hits across arXiv + Semantic Scholar + ECCC

Search queries (3): - symmetry groups AND boolean circuit complexity AND depth inequality
- group theory AND circuit depth AND inequality of order - Boolean circuits AND symmetry
group order AND depth bound

5. Empirical test harness

Multi-seed protocol: 5 seeds (11, 23, 37, 53, 71)

Sandbox: isolated subprocess, stdlib-only, ≤90s wall time per cycle **Execution:** rc=-9, elapsed=82.4s

5.1 Generated Python source

```
import random
from fractions import Fraction

def run_trial(seed: int) -> dict:
    random.seed(seed)

    def generate_circuit(depth):
        if depth == 0:
            return ['0', '1']
        inputs = generate_circuit(depth - 1)
        outputs = []
        for i in range(len(inputs)):
            for j in range(i, len(inputs)):
                outputs.append(f'OR({inputs[i]}, {inputs[j]})')
                outputs.append(f'AND({inputs[i]}, {inputs[j]})')
        return outputs

    def calculate_symmetry_group(circuit):
        # Placeholder for actual symmetry group calculation
        # For simplicity, we assume the circuit is a binary tree and its symmetry group order is 2
        depth = len(circuit) ** 0.5 + 1
        return 2 ** (int(depth) - 1)

    def measure_circuit_depth(circuit):
        # Placeholder for actual circuit depth calculation
        # For simplicity, we assume the circuit is a binary tree and its depth is log base 2 of the number of nodes
        num_nodes = len(circuit)
        return int(num_nodes ** 0.5) + 1

    n_max = 40
    instances_tested = 30
    total_metric_value = 0
    conjecture_holds_count = 0
    counterexample = ""

    for _ in range(instances_tested):
```

```

depth = random.randint(5, 40)
circuit = generate_circuit(depth)
symmetry_group_order = calculate_symmetry_group(circuit)
circuit_depth = measure_circuit_depth(circuit)

if symmetry_group_order > 2 ** circuit_depth:
    conjecture_holds_count += 1
    counterexample = f"Depth: {circuit_depth}, Symmetry Group Order: {symmetry_group_order}"

total_metric_value += symmetry_group_order

mean_metric_value = total_metric_value / instances_tested
support_fraction = conjecture_holds_count / instances_tested

return {
    "metric_name": "Symmetry Group Order",
    "metric_value": mean_metric_value,
    "instances_tested": instances_tested,
    "n_max": n_max,
    "conjecture_holds": support_fraction >= 0.8,
    "counterexample": counterexample
}

if __name__ == "__main__":
    import sys
    seeds = [int(s) for s in sys.argv[1:]] or [2**i - 1 for i in range(5, 31)]

    results = []
    for seed in seeds:
        result = run_trial(seed)
        print(f"TRIAL: {result}")
        results.append(result)

    mean_metric_value = sum(r["metric_value"] for r in results) / len(results)
    support_fraction = sum(1 for r in results if r["conjecture_holds"]) / len(results)

    if all(r["conjecture_holds"] for r in results):
        print(f"RESULT: SUPPORTED mean={mean_metric_value} std=0.0 support_fraction={support_fraction}")
    elif any(not r["conjecture_holds"] for r in results):
        first_failing_seed = next(s for s, r in zip(seeds, results) if not r["conjecture_holds"])
        print(f"RESULT: FALSIFIED counterexample={r['counterexample']} first_failing_seed={first_failing_seed}")
    else:
        print("RESULT: INCONCLUSIVE reason=unknown")

```

6. Per-seed results

(no seed-level data — test crashed before producing TRIAL: lines)

7. Test stdout (last 2KB)

(empty)

8. Critic adversarial review

Critic verdict: CHALLENGE

Critic reasoning:

skipped: test crashed before producing data

9. Final verdict & safety rail

Verdict: INCONCLUSIVE

Reasoning:

The test crashed before producing data, which means we cannot verify if the conjecture is supported or falsified according to the pre-registered support condition. | next: Re-run the test with a different set of seeds and ensure it completes successfully to verify the conjecture.

11. Audit log (LLM calls)

Total LLM calls: 10

#	Phase	Provider	Model	Tokens in	out	Latency (ms)
1	propose	ollama_remote	glm4:latest	0	0	16101
2	propose	ollama_remote	glm4:latest	0	0	19674
3	preregistration	ollama_remote	glm4:latest	0	0	9318
4	novelty	ollama_remote	glm4:latest	0	0	10083
5	novelty	ollama_remote	glm4:latest	0	0	8804
6	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	15850
7	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	8038
8	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	74699
9	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	16471
10	judge	ollama_remote	glm4:latest	0	0	101274

Totals: 0 input tokens, 0 output tokens, ~\$0.0000 cost, 280311 ms total latency. Provider mix: {'ollama_remote': 10}

(full prompt+response transcripts available in research/audit/2dafccccf1e2.jsonl)

12. Reproducibility

To reproduce this cycle:

1. Apply the test harness in section 5.1 with seeds 11,23,37,53,71
2. The Python sandbox is pure-stdlib; any Python 3.11+ should suffice
3. The full LLM transcripts are in research/audit/2dafccccf1e2.jsonl for verification of provenance
4. The pre-registration hash in section 2 commits to the test design before execution; tampering would be detectable.

Sandbox file: research/pvsnp_sandbox/test_*.py (per-cycle)

Replay tarball: research/replay/2dafccccf1e2.tar.gz (if generated)